Testēšanas plāna izstrāde

1. uzdevums. Kāpēc nepieciešama testēšana?

1.1. Kas ir programmatūras testēšana?

Programmatūras testēšana ir process, kurā pārbauda, vai programma darbojas pareizi un atbilst prasībām. Testēšanas laikā meklē kļūdas (bugus), pārbauda ievades apstrādi, rezultātu pareizumu, lietojamību un stabilitāti.

1.2. Kāpēc testēšana jāveic jau izstrādes laikā?

Testējot izstrādes laikā, kļūdas var atrast agrāk, tās ir vieglāk un lētāk labot. Tas palīdz izvairīties no situācijas, kad projekta beigās atklājas kritiskas problēmas.

1.3. Nosauc vismaz 3 problēmas, kas var rasties netestētā programmā.

- Nepareizi rezultāti vai nepareiza loģika (piem., brīdinājums lielām grupāmneparādās).
- Kļūdaina ievade netiek apstrādāta (tukši lauki, datums pagātnē), rodas kļūdas vai“salūst” lapa.
- Slikta lietojamība un nepareizs izkārtojums dažādās ierīcēs (mobilajā ekrānā laukipārklājas).

2. uzdevums. Produkta funkcionalitātes analīze

Funkcionalitāte	Lietotāja darbība	Sagaidāmais rezultāts
Rezervācija uz vārda	Ievada vārdu laukā	Ja vārds derīgs — tiek izmantots rezervācijas kopsavilkumā; ja nederīgs — kļūdas paziņojums
Cilvēku skaits (pamatuzdevums)	Ievada cilvēku skaitu (1–30)	Nepareiza skaita gadījumā — kļūda; pareizi — tiek izveidots rezervācijas teksts
Datums un laiks	Izvēlas datumu un laiku	Datums nevar būt pagātnē, laiks obligāts; pareizi — rezervācija tiek apstiprināta
Brīdinājums lielām grupām (papildus)	Ievada 8+ vai 12+ cilvēkus	Parādās brīdinājuma ziņojums (warn) un ieteikums apstiprināt pa tālruni
Rezultāta attēlošana	Nospiež “Rezervēt”	Rezultāts tiek parādīts lapā bez pārlādes (success / warn / error)

3. uzdevums. Ko nepieciešams testēt?

Testējamais aspekts	Kāpēc tas jāpārbauda
Funkcionalitāte	Jāpārliecinās, ka poga “Rezervēt” veido pareizu rezultātu un brīdinājumus, kā arī netiek rādīti nepareizi statusi.
Lietotāja ievade	Jāpārbauda, ka nederīga ievade tiek atklāta (tukši lauki, datums pagātnē, cilvēku skaits ārpus robežām).
Dizains un izkārtojums	Jāpārbauda, ka lapa ir pārskatāma, lauki nepārklājas un darbojas dažādos ekrāna izmēros.
JavaScript darbība	Jāpārbauda, ka nav kļūdu konsolē, notikumi (addEventListener) darbojas un DOM atjaunojas bez pārlādes.

4. uzdevums. Testēšanas gadījumu plānošana

Testa Nr	Ko pārbaudīsi	Lietotāja darbība	Sagaidāmais rezultāts
1	Derīga rezervācija (success)	Ievada vārdu (Anna), cilvēki=4, izvēlas šodienu vai nākotni, laiks=19:00, zona=iekšā, spiež “Rezervēt”.	Rādās success ziņojums ar kopsavilkumu.
2	Brīdinājums lielai grupai (warn)	Ievada vārdu, cilvēki=8, datums un laiks izvēlēti, spiež “Rezervēt”.	Rādās warn ziņojums par lielu grupu.
3	Brīdinājums ļoti lielai grupai (warn 12+)	Ievada vārdu, cilvēki=12, datums un laiks izvēlēti, spiež “Rezervēt”.	Rādās warn ziņojums ar stingrāku ieteikumu apstiprināt pa tālruni.
4	Kļūda: tukšs vārds	Atstāj vārdu tukšu, aizpilda pārējo, spiež “Rezervēt”.	Rādās error; vārda lauks iezīmēts kā nederīgs.
5	Kļūda: datums pagātnē / nav laika	Ievada datus, iestata datumu vakar vai neizvēlas laiku, spiež “Rezervēt”.	Rādās error; datuma/laika lauki tiek iezīmēti kā nederīgi.

5. uzdevums. Kļūdainas ievades plānošana

Ievades lauks	Iespējamā kļūdainā ievade	Kā sistēmai jāreaģē

Rezervācija uz vārda	Tukšs teksts, 1 simbols	Parādīt kļūdu, iezīmēt lauku, neizveidot rezervāciju.
Cilvēku skaits	0, -3, 100, burti	Parādīt kļūdu, iezīmēt lauku, nepāriet uz apstiprinājumu.
Datums	Nav izvēlēts vai datums pagātnē	Parādīt kļūdu, iezīmēt lauku, nepāriet uz apstiprinājumu.
Laiks	Nav izvēlēts (tukša izvēle)	Parādīt kļūdu, iezīmēt lauku, nepāriet uz apstiprinājumu.

6. uzdevums. Testēšanas plāna izvērtējums

Kā Tavs testēšanas plāns palīdzēs izveidot kvalitatīvāku produktu?

Plāns nosaka konkrētus testus (success / warn / error scenārijus), tāpēc var sistemātiski pārbaudīt gan pamatfunkciju (cilvēku skaits), gan papildu loģiku (brīdinājumi lielām grupām), kā arī validāciju. Tas palīdz ātri atrast kļūdas un nodrošina, ka lietotājs saņem pareizu, saprotamu atgriezenisko saiti.

Kad izstrādes procesā Tu veiksi testēšanu?

☑ pēc katras funkcijas izveides ☐ projekta beigās ☑ regulāri izstrādes laikā

Pamato savu izvēli:

Testēšu pēc katras funkcijas izveides un regulāri izstrādes laikā, jo tā kļūdas tiek pamanītas uzreiz. Piemēram, pēc validācijas pievienošanas uzreiz jāpārbauda tukši lauki un robežvērtības, bet pēc brīdinājumu loģikas pievienošanas — 8+ un 12+ gadījumi.